Módulo I - Análise Léxica

Especificação e Tratamento de Tokens

Fundamentos de Compiladores

UNEB

Prof. Antonio Atta

Análise Léxica – Especificação e Tratamento de Tokens
Conceitos

- **Token**

  Unidade lógica de uma linguagem dotada de significado próprio

  Palavras reservadas (**while**, **repeat**, **switch** em C), identificadores (variáveis, nomes de funções), sinais (+ , *, >, >=, etc), constantes numéricas, literais

- **Alfabeto**

  Conjunto de símbolos finito e não vazio

  Convencionalmente, usa-se o símbolo Σ para um alfabeto

  Σ_1 = { 0, 1 } alfabeto binário

  Σ_2 = { a,b,c,d } alfabeto das quatro primeiros letras do alfabeto

- **Palavra ou cadeia**

  Sequência finita de símbolos de um alfabeto

  1010 é uma cadeia do alfabeto Σ_1 = { 0, 1 }

Análise Léxica – Especificação e Tratamento de Tokens
Conceitos

- **Cadeia Vazia**

  Cadeia formada por zero símbolos e representada por ε

- **Comprimento de uma cadeia**

  É o número de símbolos do alfabeto que a cadeia contém

  A cadeia 0110 do alfabeto Σ1 = { 0, 1 } tem comprimento 4 , pois contém 4 símbolos, ou seja, |0110| = 4

  A cadeia 010101 do alfabeto Σ2 = { 01, 02 } tem comprimento 3, ou seja, |010101| = 3

  |ε| = 0

- **Potência de um alfabeto**

  Conjunto de todas as cadeias de certo comprimento

  Convencionalmente, Σ^k denota todas as cadeias de comprimento **k** do alfabeto Σ

  Se Σ = { 0, 1 }

  $\Sigma^0 = \{\varepsilon\}$ $\Sigma^1 = \Sigma = \{0,1\}$ $\Sigma^2 = \{00,01,10,11\}$

  $\Sigma^3 = \{000,001,010,011,100,101,110,111\}$

- **FECHO - Σ***

Conjunto de todas as cadeias sobre um alfabeto Σ

$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \ldots\ldots$ *(Fecho de Σ)*

$\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \ldots\ldots$ *(Fecho Positivo de Σ)*

- **Concatenação de cadeias**

Justaposição de seus símbolos – sejam x e y denotando cadeias sobre um alfabeto Σ então xy denota a concatenação, e representa a cadeia formada pelos símbolos da cadeia x seguido (justapostos) pelos símbolos da cadeia y

Seja x= 0101 do alfabeto Σ = { 0, 1 } e y = 1111 do Σ = { 0, 1 } .

Então xy = 01011111

Observar que |xy| = |x| + |y|

- **Linguagens**

Conjunto de cadeias de um determinado alfabeto

$$L \subseteq \Sigma^*$$

Exemplos de linguagem sobre o alfabeto Σ = { 0, 1 } :

A linguagem de todas as cadeias que consistem de ***n*** 0's seguidos de ***n*** 1's

L = {ε, 01,0011, 000111, ...}

A linguagem descrita pelo conjunto de cadeias de 0's e 1's com número igual de cada um deles

L = {ε, 01,10, 0011, 0101, 1001,1010, ...}

A linguagem dos números binários, cujo valor é um número primo

L = { 1, 10, 11, 101, 111, 1011, ...}

A linguagem vazia, ou seja, sem nenhuma cadeia

L = { }

A linguagem de cadeias vazias, ou seja cadeias sem símbolos

L = {ε }

A linguagem de todas as cadeias do alfabeto

L = Σ*

Um dos principais problemas associados à área de linguagens é:

Como saber se uma determinada cadeia pertence ou não a uma linguagem?

- **Linguagens Regulares - LR**
  - Na hierarquia de Chomsky corresponde ao tipo mais simples de linguagem
  - Normalmente o tipo adequado para especificar os tokens de uma linguagem de programação
  - Formalismos para a sua especificação:
    - Expressões Regulares – ER (Denotacional)
    - Autômatos Finitos – AF (Reconhecedor)
    - Gramáticas Regulares – GR (Gerador)

- **Expressões Regulares**
  - **Forma compacta e estruturada de denotar uma LR**
  - **São construções recursivas de expressões regulares menores, podendo ser definidas como se segue:**

BASE:

1. **ε** é uma ER, e *L(**ε**)* é {**ε**} , ou seja, a linguagem cujo único membro é a cadeia vazia
2. Se *a* é uma símbolo pertencente a ∑, então **a** é uma ER e *L(**a**)* = {*a*}, ou seja, a linguagem com uma cadeia de tamanho um, com a em sua única posição

- **Expressões Regulares**

INDUÇÃO:

Existem 4 partes na indução, por meio das quais ER maiores são construídas a partir das menores

1. (*r*)**|**(*s*) é uma ER denotando a linguagem *L(r)* U *L(s)*
2. (*r*)(*s*) é uma ER denotando a linguagem *L(r)L(s)*
3. (*r*)* é uma ER denotando (*L(r)*)*
4. **(***r***)** é uma ER denotando *L(r)*. Esta regra diz que podemos acrescentar pares de parênteses adicionais em torno de ER's, sem alterar a linguagem que elas denotam

OBS: Nas ER's sem o uso dos parênteses, o operador * tem maior precedência e é associativo à esquerda; a concatenação possui segunda maior precedência e é associativo à esquerda; finalmente, | possui a precedência mais baixa e também é associativo à esquerda

- **Expressões Regulares – Exemplos**
  - A ER a|b denota a linguagem {*a, b*}
  - (a|b)(a|b) denota {*aa, ab, ba, bb*}
  - a* denota a linguagem consistindo de todas as cadeias com zero ou mais a's, ou seja, {*ε, a, aa, aaa, ...*}
  - (a|b)* denota a linguagem {*ε, a, b, aa, ab, ba, bb, aab, baa, bba, ...*}
  - a|a*b denota a linguagem {*a, b, ab, aab, aaab, ...*}

- **Exercícios**

Apresente Expressões Regulares para as seguintes linguagens sobre o alfabeto {a, b}:

1. Todas as palavras que iniciam por **b**, seguido por zero ou mais **a**
2. Todas as palavras contendo **aa** como subpalavra
3. Todas as palavras contendo exatamente dois **b**
4. Todas as palavras que terminam com **aa** ou **bb**
5. Todas as palavras que não possuem dois **a** consecutivos

- **Mais exercícios**

Apresente ERs para as seguintes linguagens:

1. Conjunto de palavras sobre {a,b,c} contendo ao menos um **a** e ao menos um **b**

2. Conjunto de palavras sobre {0,1} tal que cada par de **0**s adjacentes aparece antes de qualquer par de **1**s

3. Identificadores da linguagem Pascal que são compostos por uma letra (a...z) ou sublinhado (_) seguido por qualquer combinação de letras, sublinhados ou dígitos (0...9).

- **Mais exercícios**

Apresente ERs para as seguintes linguagens:

4. $\{ w \in \{a, b\}^* \mid |w| >= 3 \}$

5. $\{ w \in \{a, b\}^* \mid$ w começa com a e tem tamanho par$\}$

6. $\{ w \in \{a, b\}^* \mid$ w tem um número par de a´s$\}$;

- **Definições regulares**

  Nomes dados a certas expressões regulares para tornar a notação mais conveniente

*letra_*	→	A | B | C | ... | Z| a | b | c |...| z | _
digito	→	0 | 1 | 2 | ... | 9
identif	→	*letra_* (*letra_* | *digito*)*

- Qual seria uma possível Definição Regular para uma constante numérica inteira, real ou exponencial?

  Exemplos: 5, 5.25, 5.25E-3

- **Autômatos Finitos - AF**
  - Os AF's são reconhecedores de cadeias de uma linguagem regular
  - Eles simplesmente dizem "sim" ou "não" ao fato de uma cadeia pertencer a uma LR
  - São sistemas de estados finitos
  - Para cada caracter da cadeia avaliada um AF transita para um (ou mais) estado(s) de um conjunto pré-definido de estados
  - Caso o AF consiga "consumir" todos os caracteres da cadeia e termine em um estado de aceitação, o AF sinaliza que a cadeia pertence a linguagem

- **Autômatos Finitos – Definição**

Um autômato finito (AF) é uma quíntupla $M=(Q,\Sigma,\delta,q_0,F)$, onde:

- **Q: conjunto finito de estados;**
- **Σ : um alfabeto;**
- **q_0: estado inicial;**
- **F: conjunto de estados finais;**
- **δ : função de transição de estados definida como:**

$$\delta(q_p, x_k)=q_j$$

Indica para <u>cada</u> estado e <u>cada</u> letra do alfabeto para qual estado se deve seguir (isto é, qual é o próximo estado)

Um autômato finito pode ainda ser representado sob a forma de *grafo orientado*, com os estados representando os vértices e as arestas sendo rotuladas com os caracteres do alfabeto, ou através de uma *tabela de transições*

Exemplo:

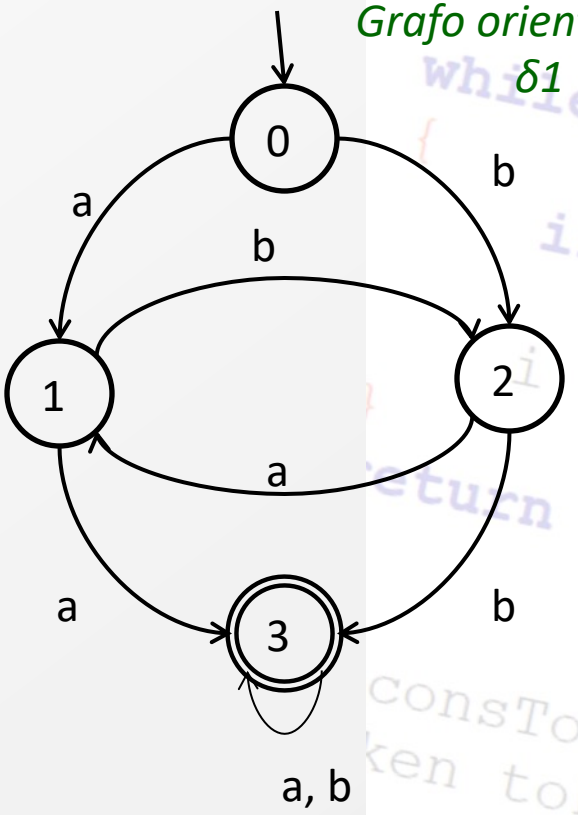
Seja L_1={w | w possui **aa** ou **bb** como subpalavra}. Um AF possível para L_1 é:

M_1=({a,b}, {0,1,2,3}, δ_1, 0, {3}), onde:

δ_1	a	b
0	1	2
1	3	2
2	1	3
3	3	3

Tabela de Transição de δ1

Grafo orientado de δ1



- **Condições de parada de uma AF:**

1. A cadeia de entrada é toda processada e o autômato se encontra em um estado de aceitação (final) → o AF pára e a cadeia é aceita;

2. A cadeia de entrada possui um caracter para o qual não existe transição a partir do estado em que o AF se encontra (seja ele de aceitação ou não) → o AF pára e a cadeia não é aceita;

3. A cadeia de entrada é toda processada e o autômato se encontra em um estado que não é de aceitação (final) → o AF pára e a cadeia não é aceita;

- **Exercício**

Qual um possível AF para a linguagem L_2 abaixo?

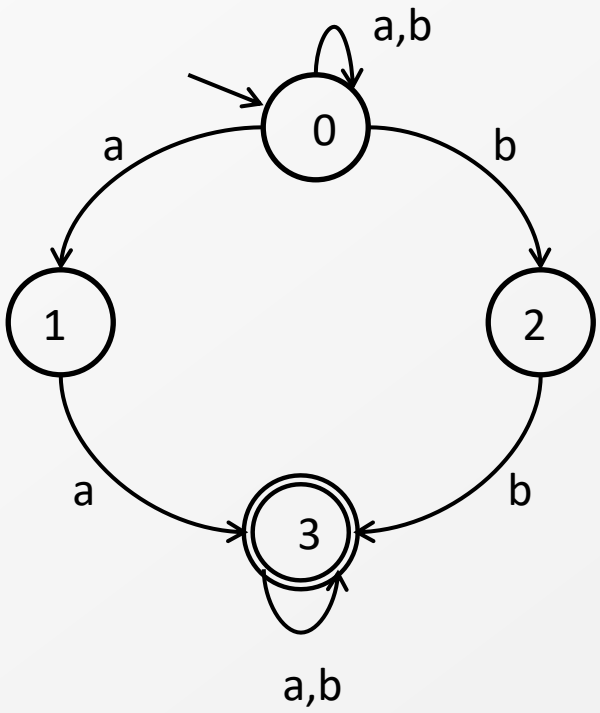
L_2={w | w possui um número par de **a** e **b**}

Qual uma possível ER para esta linguagem?

- **Autômatos Finitos Não Determinístico – AFN**
  - AF determinísticos são computacionalmente mais adequados mas, às vezes, difíceis de serem concebidos
  - Uma generalização dos AF's possível permite o não determinismo na transição dos estados durante o processamento dos caracteres da cadeia de entrada
  - Esta generalização facilita a concepção de AF's e é uma importante ferramenta para a obtenção automática de autômatos

Exemplo:

Para a linguagem L_1={ w | w possui **aa** ou **bb** como subcadeia} anterior, um AFN possível seria:



- **Exercício**

Qual um possível AF para a linguagem L_3 abaixo?

L_3={w | w possui **aaa** como sufixo}

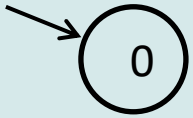
Encontre agora um AFN equivalente para L_3

- **Autômatos Finitos com Movimentos Vazios - AFε**
  - Constituem uma generalização dos AFN
  - Um movimento vazio é rotulado como ε e corresponde a uma transição sem consumo de caracteres da cadeia de entrada
  - Esse modelo facilita algumas construções e demonstrações com autômatos
  - Exemplo:

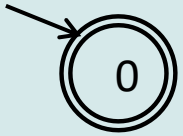
- **Obtenção de um AF determinístico a partir de uma ER**
  - **Passo 1: Construção do AFε correnpondente – Algoritmo de Thompson**

BASE:

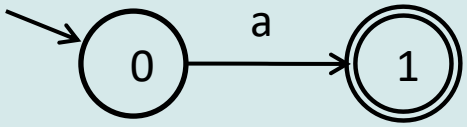
1. Se *r*=Φ então o AF correspondente é:



2. Se *r*=**ε** então o AF correspondente é:

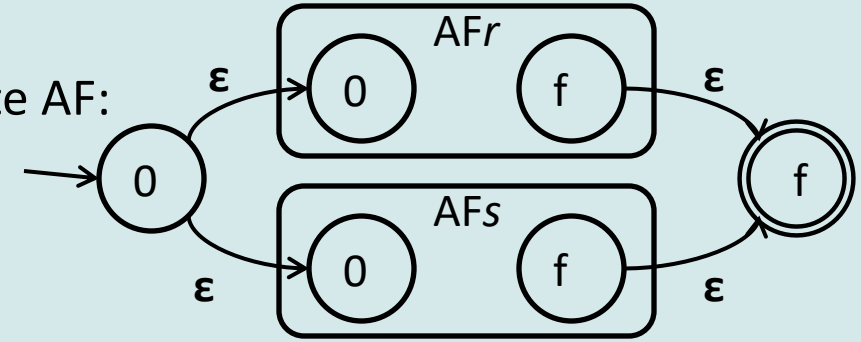


3. Se *r*=**a** então o AF correspondente é:

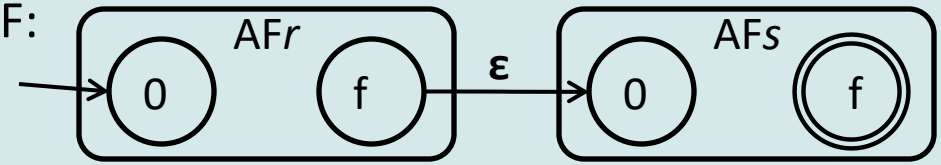


- **INDUÇÃO:**

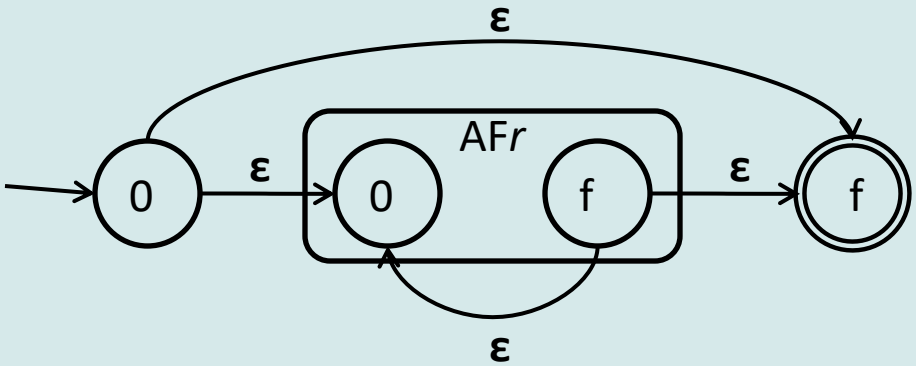
1. *(r)*|*(s)* possui o seguinte AF:



2. *(r)(s)* possui o seguinte AF:



3. *r** possui o seguinte AF:



- **Exemplo**

Usando o algoritmo de Thompson, construir o AFε para **(a|b)*abb**

- **Passo 2: Transformação do AFε em um AF determinístico – Algoritmo dos Sub-conjuntos**

Função	Descrição
Fecho-ε (q)	Fechamento vazio a partir de um estado **q** – conjunto de estados atingíveis por movimentos vazios a partir do estado **q**
Fecho-ε (Q)	Fechamento vazio a partir de um conjunto de estados Q – conjunto de estados atingíveis por movimentos vazios a partir de qualquer um dos estados pertencentes a Q
Movimento (Q, a)	Conjunto de estados atingíveis por transição ao ler o caracter a a partir de qualquer um dos estados pertencentes a Q

- **Minimizando os estados de um AF**
  - Visa obter um AF mais eficiente em termos de ocupação de espaço de memória
  - Algoritmo:

    1. Crie uma partição inicial ∏ com dois grupos do AF: um grupo formado pelos estados de aceitação e outro grupo formdo pelos estados restantes
    2. Para cada grupo *G* de ∏,

       Particione *G* em subgrupos tais que dois estados *s* e *t*

       estejam no mesmo subgrupo se e somente se para todos os símbolos de entrada *a*, os estados *s* e *t* tenham transições sob a para estados de um mesmo grupo de ∏

       Substitua G em ∏ pelo conjunto de todos os subgrupo formados
    3. Repita o passo 2 até que ∏ não se modifique mais

- **Minimizando os estados de um AF (cont.)**

4. Escolha um estado em cada grupo de ∏ como representante para esse grupo. Tais representantes serão os estados do AF mínimo assim constituído:

   (a) O estado inicial do AF mínimo é o representante do grupo contendo o estado inicial do AF original

   (b) Os estados de aceitação do AF mínimo são os representantes daqueles grupos que contém estados de aceitação do AF original

   (c) Para qualquer caracter *a*, se existe uma transição sob *a* de *s* para *t* no AF original e no AF mínimo *r* é o representante do grupo ao qual *t* pertence, então existe uma transição sob *a* de *s* para *r* no AF mínimo

- **Gramáticas Regulares**

Uma gramática regular (GR) é uma quadrupla $G=(V_T, V_N, P, S_i)$, onde:

- **V_T : conjunto finito de símbolos *terminais* da linguagem; corresponde ao alfabeto da linguagem.**
- **V_N : conjunto finito de símbolos *não-terminais* da linguagem; são símbolos variáveis que podem ser derivados em composições mais específicas da linguagem (contendo combinações de símbolos terminais e não-terminais) a partir das regras de produção P.**
- **P: conjunto de regras de produção da linguagem; indicam como cada símbolo não-terminal da linguagem pode ser substituído (ou derivado) em combinações (ou sequências) de símbolos terminais e/ou não-terminais no processo de geração de cadeias válidas da linguagem**
  - **Formato: $X \rightarrow Y$, onde $X \in V_N$ e $Y \in (V_N \times V_T)$**
- **S_i : corresponde ao símbolo não terminal de partida.**

- **Gramáticas Regulares - Exemplos**

1. Linguagem L_1 de todas as cadeias de 0's e 1's que não contenham dois 0's seguidos:
   - L_1 : ({S, A}, {0, 1}, P, S) com P: $\begin{cases} S \rightarrow 0A \mid 1S \mid \varepsilon \\ A \rightarrow 1S \mid \varepsilon \end{cases}$

2. Linguagem L_2 de todas as cadeias de 0's e 1's que terminem em 1:
   - L_2 : ({S, A}, {0, 1}, P, S) com P: $\begin{cases} S \rightarrow 1A \mid 0S \\ A \rightarrow 1A \mid 0S \mid \varepsilon \end{cases}$

3. Linguagem L_3 de todas as cadeias de a's e b's que contenham 'aa' como subcadeia:
   - L_3 : ({S, A, B}, {a, b}, P, S) com P: $\begin{cases} S \rightarrow aA \mid bS \\ A \rightarrow aB \mid bS \\ B \rightarrow aB \mid bB \mid \varepsilon \end{cases}$

- **Gramáticas Regulares - Propriedades**

As gramáticas regulares (GR) correspondem a formalismos geradores de cadeias válidas de uma linguagem regular.

O processo de geração de cadeias de uma linguagem a partir de uma GR é conhecido como *derivações*.

A partir do símbolo inicial da GR (símbolo de partida) iniciamos as derivações substituindo os símbolos não-terminais na cadeia gerada utilizando uma das regras de produção associada ao símbolo não-terminal que será substituído.

O processo derivação se repete até que só existam símbolos terminais na cadeia gerada.

- **Gramáticas Regulares - Tipos**

Seja $G=(V_T, V_N, P, S_i)$, uma gramática e sejam A e B elementos de V_N e e w uma palavre de V_T^* . Então G é uma:

a. **Gramática Linear à Direita (GLD) se todas as regras de produção são da forma: $A \rightarrow wB$ ou $A \rightarrow w$;**

b. **Gramática Linear à Esquerda (GLE) se todas as regras de produção são da forma: $A \rightarrow Bw$ ou $A \rightarrow w$;**

c. **Gramática Linear Unitária à Direita (GLUD) se se todas as regras de produção são como na GLD e, adicionalmente $|w| \leq 1$**

d. **Gramática Linear Unitária à Esquerda (GLUE) se se todas as regras de produção são como na GLE e, adicionalmente $|w| \leq 1$**

- **Gramáticas Regulares - Exercícios**

1. **Apresente Gramáticas Regulares para as seguintes linguages regulares:**

   a. **Linguagem de palavras sobre o alfabeto {0, 1} contendo um número par de zeros**

   b. **Linguagem de palavras sobre o alfabeto {a, b} que contêm exatamente um 'b'**

2. **Descreva a relação que existe entre AFDs e GLUDs**